

binky

Technical Reference

A description of what the code actually does, file by file. Reflects the codebase as of release v1.0.0.

1. Overview

binky is a single-purpose Android app: log beverages and time brews. Built in Flutter, targets Android only. All data is stored locally on the device in encrypted app-private storage. The app does not declare the INTERNET permission, so it cannot make network requests.

Concern	Choice
Language	Dart (Flutter 3.22+ / Dart 3.4+)
Platform	Android only (minSdk 21, target follows Flutter default)
Architecture	Single-Activity Flutter app, no custom native channels
State management	Plain ValueNotifiers + static service classes — no Provider / Riverpod / Bloc
Persistence	flutter_secure_storage (AES-256-GCM via Android Keystore + EncryptedSharedPreferences)
Background work	flutter_foreground_task (separate Dart isolate) + scheduled OS notification
Audio	audioplayers with fresh AudioPlayer per call; MEDIA stream, gainTransientMayDuck focus

2. File layout

All Dart source lives in app/lib/. Twelve files total.

File	Role
main.dart	App entry, root MaterialApp, HomeScreen, active-brew banner widget, lifecycle observer install
storage.dart	SecureStore wrapper, Drink + LogEntry models, DrinksStore + LogStore (CRUD), one-shot SharedPreferences → secure-storage migration
screens.dart	DrinkScreen, TimerScreen, SummaryScreen, weekly chart
active_brew.dart	ActiveBrewState + ActiveBrew controller (the timer state machine), lifecycle-sync observer
alarm.dart	In-app audioplayers wrapper, OS scheduled completion

File	Role
	notification (flutter_local_notifications)
brew_service.dart	Foreground-service isolate entypoint and TaskHandler — plays audio while phone is locked
settings.dart	Settings load/save + SettingsScreen UI; notifiers for theme, kettle, custom sound, keep-screen-on
history.dart	HistoryScreen — list + swipe-delete + edit dialog + CSV export trigger
drinks_editor.dart	DrinksEditorScreen + new/edit drink form
export.dart	LogExporter.shareCsv() — CSV generation + OS share sheet
diagnostics.dart	In-process log ring buffer (200 entries) + DiagnosticsScreen
splash_screen.dart	Brief launch splash before HomeScreen

Android platform code is the standard Flutter scaffold (MainActivity.kt is a 5-line FlutterActivity subclass). All app logic is in Dart.

3. Data model

Two-level drink hierarchy: fixed Category (5 of them) → user-editable Drink sub-types.

3.1 DrinkCategory

Fixed list: Hot Tea, Cold Tea, Coffee, Soda, Bog Standard Water. Each has a Material icon, default volume, brewable flag, and default brew-time presets. Not user-editable.

3.2 Drink

Per-category sub-type. Fields: stable int id, type (matching the category name), description, defaultVolume (oz), brewable, brewTimes (preset minutes), isDefault. DrinksStore.upsert enforces exactly-one-default-per-category.

Seeded list ships 13 sub-types on first launch; defaults are Green Tea, Triple Berry, Cream & Sugar, Diet Pepsi, Water.

3.3 LogEntry

One recorded beverage. Fields: id (microsecondsSinceEpoch at log time), timestamp, drinkId, type (denormalized), description (denormalized), volume, notes.

type/description are denormalized intentionally — deleting a drink doesn't invalidate historical entries; they keep their original labels.

3.4 ActiveBrewState

The single in-progress brew, if any. Fields: endsAt, originalDuration, appBarTitle, alarmTitle, alarmBody, doneLabel, drinkIdToLog (nullable — null for kettle), drinkTypeForLog, drinkDescriptionForLog, volume, notes.

Brews and kettle timers share the same type; they differ only in whether `drinkIdToLog` is set.

4. Storage

All persistent state lives in `flutter_secure_storage` (Android Keystore + EncryptedSharedPreferences — AES-256-GCM). `SecureStore` in `storage.dart` is a thin wrapper that mirrors the `SharedPreferences` API (`getString/setString/getInt/setInt/getStringList/setStringList`) on top of the secure backing store.

Key	Contents
<code>drinks_v2</code>	JSON array of <code>Drink</code> objects (with <code>isDefault</code> field)
<code>brew_log</code>	List of JSON-encoded <code>LogEntry</code> objects
<code>theme_mode</code>	"system" "light" "dark"
<code>kettle_minutes</code>	1–30, default 5
<code>active_brew_v1</code>	JSON-encoded <code>ActiveBrewState</code> , or absent if no brew
<code>custom_alert_sound_path</code>	Absolute path to the user-chosen audio file (copied into app-private storage)
<code>keep_screen_on_during_brew</code>	"true" / "false", default true
<code>_migrated_to_secure_v1</code>	One-shot migration flag (so the <code>SharedPreferences</code> → secure migration runs at most once per install)

Migration from pre-encryption storage

`SecureStore.migrateFromSharedPreferences()` runs first on every cold start. If the migration flag is absent, it reads every known key from the old `SharedPreferences` instance, writes it into secure storage, then sets the flag. The old `SharedPreferences` entries are also cleared. Idempotent — second-and-later launches see the flag and return immediately.

5. Reactive state — global notifiers

No dependency-injection framework. Screens read via `ValueListenableBuilder`; mutations go through service methods that update both the persisted store and the notifier.

Notifier	Type	Source of truth
<code>drinksNotifier</code>	<code>List<Drink></code>	<code>storage.dart</code>
<code>activeBrewNotifier</code>	<code>ActiveBrewState?</code>	<code>active_brew.dart</code>
<code>themeModeNotifier</code>	<code>ThemeMode</code>	<code>settings.dart</code>
<code>kettleMinutesNotifier</code>	<code>int</code>	<code>settings.dart</code>
<code>customAlertSoundPathNotifier</code>	<code>String?</code>	<code>settings.dart</code>
<code>keepScreenOnDuringBrewNotifier</code>	<code>bool</code>	<code>settings.dart</code>

6. Screens and navigation

Navigation strategy:

- **Brew** → **Timer** uses `pushReplacement` so backing out of `TimerScreen` lands on `Home`, not back on `DrinkScreen`.
- **Kettle** → **Timer** uses plain `push` so popping returns to `DrinkScreen`, where `Brew` is one tap away. `_onDone` returns true via `Navigator.pop(true)` so the caller can show a "Water's hot" `SnackBar`.
- `TimerScreen.dispose()` does NOT call `ActiveBrew.stop()`; backing out leaves the brew running. Only `Cancel` and `Done` resolve a brew.
- `Cancel/Done` in `TimerScreen` pop FIRST, then fire `ActiveBrew.stop()/complete()` via `unawaited()` to avoid the `ValueListenableBuilder` rebuilding a black scaffold with null brew before pop completes.

7. The brew system

The most complex piece. Two isolates cooperate to keep the alert ringing across foreground, background, and locked states.

7.1 States

State	Check	Meaning
Idle	<code>ActiveBrew.current == null</code>	No brew running
Running	<code>current != null && !current.expired</code>	Counting down
Expired	<code>current != null && current.expired</code>	Hit zero; user hasn't tapped <code>Done</code> or <code>Cancel</code> yet

`expired` is a computed property: `endsAt.isBefore(DateTime.now())`.

7.2 Transitions

From	To	Trigger	What happens
Idle	Running	<code>ActiveBrew.start(...)</code>	Acquire wake lock (if setting is on). Schedule OS completion notification (<code>inexactAllowWhileIdle</code>). Start foreground service. Persist <code>ActiveBrewState</code> . Emit on <code>activeBrewNotifier</code> .
Running	Running	(passive)	"expired" flips true when <code>DateTime.now()</code> crosses <code>endsAt</code> . Service's <code>onRepeatEvent</code> (1s) sees it; main's banner/ <code>TimerScreen</code> tickers see it.
Running	Idle	<code>ActiveBrew.stop()</code>	Cancel OS notification, stop foreground service, persist null, release wake lock. <code>_expiryHandled</code> is NOT reset here (only <code>start()</code> resets it).
Expired	Idle	<code>ActiveBrew.complete()</code>	Write <code>LogEntry</code> if <code>drinkIdToLog</code> is set, then same teardown as <code>stop()</code> .

7.3 The foreground service

`flutter_foreground_task` runs `brew_service.dart`'s `BrewTaskHandler` in a SEPARATE Dart isolate. The service is configured in `main.dart` with channel ID `brew_in_progress_v1`, LOW importance, onlyAlertOnce, and a 1-second repeat tick. Required because Android pauses Flutter's frame callbacks when the device locks; without the service, the in-app audio path never reaches the audio call.

The service is started in `ActiveBrew._startBrewService()`, keyed on serviceId 256. State that the isolate needs (`endsAt`, `customSoundPath`) is round-tripped through `FlutterForegroundTask.saveData` before `startService` is called.

Android 14+ requires a `foregroundServiceType` for any foreground service. The manifest declares `mediaPlayback` — a defensible choice because the service's job is to play the brew-complete sound.

7.4 Two audio paths

Both paths use audioplayers (a fresh `AudioPlayer` per call — a static player loses audio focus after the first play and silently no-ops thereafter). They differ only in which isolate they run in.

Path	Isolate	Fires when
<code>Alarm.foregroundAlert()</code>	main	App is RESUMED at expiry
<code>BrewTaskHandler._fireExpiry()</code>	foreground service	App is paused / backgrounded / locked at expiry

Audio context settings (both isolates)

- `contentType`: music
- `usageType`: media — plays on the media stream so the user's media-volume rocker controls it. Routing to notification/alarm stream made the sound inaudible on devices where the user had silenced that stream.
- `audioFocus`: `gainTransientMayDuck` — asks other audio (e.g. the user's music) to lower volume briefly; focus is released when our ~1-second sound finishes and the other app returns to full volume. Plain gain caused music apps to stop and never resume.

7.5 Double-ding suppression (lifecycle handshake)

Both isolates have independent expiry detectors. Without coordination, both fire audio when the app is open and unlocked at expiry — a double ding.

Fix: a `_BrewLifecycleSync WidgetsBindingObserver` in `active_brew.dart` writes the main isolate's resumed/paused state into `FlutterForegroundTask` shared data under key `main_isolate_resumed`. The service reads this flag at the top of `_fireExpiry`:

- Flag is **true** (app resumed): the service sends `"brew_expired_main_will_alert"` to main (logged for diagnostics) and updates the persistent notification to "Brew complete", but skips its own audio and haptics. Main's banner / `TimerScreen` tickers fire `handleExpiry` within ~1s.
- Flag is **false** (paused/backgrounded/locked): existing path. Service sends `"brew_expired"` to main, vibrates three times, plays audio. When main later resumes, the message handler has already called `markExpiryHandledByService()` which set `_expiryHandled = true`, so main's banner `Timer` sees the flag and skips.

Race protection: `_expiryHandled` is set to true and reset to false ONLY by `start()` (for the next brew). `stop()` and `complete()` do NOT reset it — doing so previously opened a ~80ms race where the home-banner Timer re-fired `handleExpiry` between the flag clear and state clear.

7.6 The OS scheduled notification

`Alarm.scheduleCompletion()` uses `flutter_local_notifications` with:

- Channel: `brew_complete_v8` (high importance, with bundled WAV bound directly via `RawResourceAndroidNotificationSound`).
- Fixed notification id 9001 — reschedules replace, cancel always works without tracking per-brew ids.
- `androidScheduleMode`: `inexactAllowWhileIdle` — no exact-alarm permission required; drift is typically seconds (worst case ~15 min on heavily throttled devices). Acceptable for a brew timer; the foreground service is the primary alert path anyway.

Cancelled by `ActiveBrew.handleExpiry()` (foreground path) and by `stop()/complete()` in all cases.

7.7 Wake lock

`ActiveBrew._enableWakeLock()` calls `wakelock_plus`, which sets the Android window's `FLAG_KEEP_SCREEN_ON`. Not a true CPU wake lock — just keeps the screen on.

Gated by `keepScreenOnDuringBrewNotifier` (Settings → Keep screen on while brewing). When the toggle is off, `_enableWakeLock` logs "wake lock SKIPPED" and no-ops. Released in `stop()/complete()` regardless.

7.8 Persistence and restoration

`ActiveBrew._persist()` writes to SecureStore key `active_brew_v1` on every transition. `ActiveBrew.load()` runs once at app startup. On load:

- No persisted state → idle.
- Persisted state with `endsAt` more than 1 hour in the past → discarded as stale.
- Otherwise → restored. Wake lock re-acquired if the brew is still running.

7.9 Brew vs. Kettle

Same `ActiveBrew` type; differences are entirely in calling-side wiring.

Aspect	Brew	Kettle
<code>drinkIdToLog</code>	set to the current drink	null
Done label	Done — log it	Brew now
Done action	<code>complete()</code> writes a <code>LogEntry</code>	<code>complete()</code> no-ops on logging
Navigation	<code>pushReplacement</code> — <code>DrinkScreen</code> dropped	push — <code>DrinkScreen</code> stays underneath
Post-completion	Lands on Home	Returns to <code>DrinkScreen</code> with "Water's hot" <code>SnackBar</code>

8. Settings & diagnostics

SettingsScreen sections: Appearance, Brew screen (keep-screen-on toggle), Kettle (slider), Alert sound (radio + file picker + preview), Diagnostics.

Diagnostics class is a 200-entry ring buffer of timestamped DiagEntry records. Every meaningful operation (brew start/stop/complete, audio attempt, audio context change, OS notif schedule/cancel, foreground service start/stop, wake lock toggle, expiry handler decisions) logs here. DiagnosticsScreen renders the buffer with Copy + Clear buttons.

Two test buttons in the diagnostics view:

- **Simulate in-app alert** — calls `Alarm.foregroundAlert()` directly.
- **Test OS notification** — calls `Alarm.testFireCompletionNotification()`, which posts notif id 9001 immediately to verify channel sound + permissions are working in isolation.

9. Theming

Material 3 with `ColorScheme.fromSeed(seedColor: Color(0xFF6B4423))` — coffee brown. Light and dark variants both generated from the same seed. `themeMode` is driven by `themeModeNotifier` (default `ThemeMode.system`). The BinkyApp widget wraps `MaterialApp` in a `ValueListenableBuilder<ThemeMode>` so changes rebuild the tree on `Settings.setThemeMode()`.

10. Permissions

Permission	Justification	Runtime prompt?
POST_NOTIFICATIONS	Brew-complete OS notification + persistent "Brew in progress" foreground-service notification	Yes, on first brew start (Android 13+)
VIBRATE	Haptic pulses at brew completion	No (normal permission)
FOREGROUND_SERVICE	Needed to start a foreground service	No
FOREGROUND_SERVICE_MEDIA_PLAYBACK	Android 14+ requirement; matches the service's declared type (mediaPlayback)	No
WAKE_LOCK	Paired with the foreground service to keep CPU active long enough to fire audio while locked	No

Permissions explicitly NOT declared

- `INTERNET` — the app cannot make any network call. This is verified by `adb shell pm dump com.binkyapp | grep INTERNET` returning nothing.
- `ACCESS_NETWORK_STATE` — irrelevant without `INTERNET`.
- `READ_EXTERNAL_STORAGE` / `WRITE_EXTERNAL_STORAGE` / `READ_MEDIA_*` — CSV export uses the app's own temp directory + OS share sheet. Custom-sound picking

uses the system file picker (temporary URI access, copied into app-private storage immediately).

- `USE_EXACT_ALARM` / `SCHEDULE_EXACT_ALARM` — OS notification uses `inexactAllowWhileIdle`.
- `RECEIVE_BOOT_COMPLETED` — alarms / foreground service don't survive reboot; brew durations are minutes, not hours.

Manifest also sets `android:allowBackup="false"` and `android:usesCleartextTraffic="false"`.

11. Build configuration

`android/app/build.gradle.kts`:

- `applicationId = "com.binkyapp"`, `namespace = "com.binkyapp"`
- `compileSdk = flutter.compileSdkVersion (~35)`, `minSdk = 21`, `targetSdk = flutter.targetSdkVersion`
- `isCoreLibraryDesugaringEnabled = true` (required by `flutter_local_notifications`), `desugar_jdk_libs:2.1.4`
- Release buildType: `isMinifyEnabled = true (R8)`, `isShrinkResources = true`, ProGuard rules from `proguard-rules.pro`
- Signing config reads from `android/key.properties` if present, falls back to debug keystore otherwise (`key.properties` is gitignored)

proguard-rules.pro keeps:

- All `io.flutter.**` classes (engine + plugin runtime)
- `com.dexterous.**` (`flutter_local_notifications` uses reflection)
- `GeneratedPluginRegistrant`
- Enum reflection for serialized notification responses
- CRITICAL: `-keepattributes Signature` + `-keep com.google.gson.reflect.TypeToken` — without these, R8 strips the generic type metadata that Gson uses internally inside `flutter_local_notifications`, and the notification scheduling chain throws `PlatformException` in release builds only.

res/raw/keep.xml

Tells the R8 resource shrinker NOT to strip `res/raw/brew_complete.wav`. Without this, the bundled chime was stripped from release APKs because the OS notification channel references it via name string (not directly inspectable to R8).

Dart obfuscation

Release build adds: `flutter build appbundle --release --obfuscate --split-debug-info=build\symbols`

The `build/symbols/` directory (per-architecture `.symbols` files) is what you'd use to de-obfuscate crash reports. Keep it locally per release; don't commit.

12. Dependencies

Production deps. None make network calls.

Package	Version	Purpose
shared_preferences	^2.2.3	Kept solely for the one-shot migration to secure storage
flutter_secure_storage	^9.2.2	AES-256-GCM encrypted local store (Android Keystore + EncryptedSharedPreferences)
audioplayers	^6.1.0	Playback of the brew-complete WAV (bundled or user-chosen)
share_plus	^9.0.0	OS share sheet for CSV export
path_provider	^2.1.3	Temp directory for CSV file + app-private dir for custom sound copy
file_picker	^8.0.7	System file picker for custom alert sound (pinned to stay compatible with share_plus 9.x's web dep)
wakelock_plus	^1.2.8	Window FLAG_KEEP_SCREEN_ON during brew (gated by setting)
flutter_local_notifications	^17.2.3	OS-scheduled completion notification (high-importance channel with bundled sound)
timezone	^0.9.4	Required by flutter_local_notifications for zoned scheduling
flutter_foreground_task	^8.10.0	Foreground service that keeps a Dart isolate alive while a brew is running

Dev dependencies

- flutter_lints ^4.0.0
- flutter_launcher_icons ^0.14.1 — generates Android mipmap icons from a source PNG
- flutter_native_splash ^2.4.1 — generates the launch splash background and icon

13. Behavior under unusual conditions

Condition	Behavior
App backgrounded mid-brew	Main isolate pauses; foreground service tick continues. Lifecycle observer writes main_isolate_resumed=false. Service is the alert path.
App swiped from recents mid-brew	Process killed. Foreground service survives (it's its own service). Audio still fires at expiry via the service isolate. On next launch, ActiveBrew.load() rehydrates state from secure storage.
App force-stopped from Settings	Process killed AND foreground service torn down AND OS alarms cleared. Persisted state survives. On next launch the banner shows the expired-brew state and the user can resolve manually. Worst case: silent miss.

Condition	Behavior
Phone reboot mid-brew	Same as force-stop. No RECEIVE_BOOT_COMPLETED.
Notification permission denied	OS notification doesn't display, but the foreground service still runs (it's a system service, not a notification). The persistent "Brew in progress" banner is still required to show — Android refuses to start a foreground service without the runtime permission on Android 13+. Net effect: no brew can start.
Two brews started in quick succession	DrinkScreen._confirmReplacelfRunning() shows a confirmation dialog. Keep Running aborts; Replace lets ActiveBrew.start internally call stop() first.
Brew expires >1 hour in the past during app load	Discarded as stale; user sees a clean slate.
Drink deleted while a brew for it is running	complete() falls back to the type/description denormalized in ActiveBrewState; the log entry still gets written with the original labels.
CSV export with zero entries	CSV with just the header row. Share sheet still opens.
Audio context "stuck" from a prior misconfiguration	_ensureAudioContext() is idempotent and explicitly resets the global AudioPlayer.global context to MEDIA / MUSIC / TRANSIENT_DUCK the first time audio is needed in the process.
Music playing in another app when brew expires	Music ducks briefly (gainTransientMayDuck), brew sound plays, music returns to full volume.

14. Known limitations / explicitly deferred

These are conscious decisions, not bugs:

1. **No reboot survival.** Foreground service + OS alarms are cleared on phone reboot. Brew durations are minutes, not hours.
2. **No notification action buttons.** Cancelling a running brew requires opening the app. Adding Cancel/Snooze on the persistent notification would be ~30 lines but isn't implemented.
3. **Single concurrent brew.** Multi-brew would need a list UI, per-brew service correlation, and more state — out of scope for this app.
4. **No cloud sync, no account, no backup.** Intentional. CSV export covers data portability. allowBackup="false" means even Android's adb-backup can't extract the database.
5. **Android only.** Flutter is cross-platform but iOS-specific notification config, foreground-service equivalents, and audio focus haven't been wired up.
6. **No internationalization.** All strings are English literals.
7. **No automated tests.** Logic-heavy classes (ActiveBrew, DrinksStore, LogStore) are pure Dart and would be straightforward to test.

8. **Dart obfuscation symbols are local-only.** Save build/symbols/ somewhere safe per release — without them, Play Console crash reports are unreadable.

15. Build & deploy quick reference

Day-to-day development

`flutter run`

Static analysis

`flutter analyze`

Generate launcher icons

`dart run flutter_launcher_icons`

Release AAB for Play Store

`flutter clean`

`flutter pub get`

`flutter analyze`

`flutter build appbundle --release --obfuscate --split-debug-info=build\symbols`

Output lands in build/app/outputs/bundle/release/app-release.aab. Bump version in pubspec.yaml for every Play upload — the build code (the number after +) must be strictly greater than the previous upload or Play will reject it.

16. Repo layout

D:\ProjectFiles\Binky\binky\

- PRIVACY_POLICY.md — public-facing, host before Play submission
- PLAY_LISTING.md — copy-paste source for Play Console
- TECHNICAL.md — markdown source for this document
- binky - Technical Reference.docx — this document
- binky - User Guide.docx — user-facing guide
- binky - Play Store Deployment Guide.docx — deployment checklist
- app/ — Flutter project
 - app/lib/ — all Dart source (12 files)
 - app/assets/sounds/brew_complete.wav — default chime
 - app/android/app/build.gradle.kts — signing, R8, applicationId
 - app/android/app/proguard-rules.pro
 - app/android/app/src/main/res/raw/brew_complete.wav
 - app/android/app/src/main/res/raw/keep.xml — R8 resource-keep directive
 - app/android/app/src/main/AndroidManifest.xml — permissions, service entry
 - app/pubspec.yaml — deps, version, launcher_icons config